Unidad 1
Control de versiones con Git

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026

El problema: versionar a mano

tp_final.zip · tp_final_v2.zip · tp_final_v2_corregido.zip

- ¿Cuál era la versión que andaba?
- Toqué algo y dejó de compilar, pero no sé qué toqué.
- Le pasé el proyecto a mi compañero, los dos lo modificamos, y ahora hay dos proyectos distintos.

Un sistema de control de versiones resuelve tres cosas: **historial confiable**, **poder volver atrás con precisión** y **trabajar de a varios sin pisarse**.

Repositorio local y repositorio remoto

Git

- Programa que gestiona el historial.
- Vive en tu máquina y funciona sin internet.
- Guarda todo en la carpeta oculta .git.

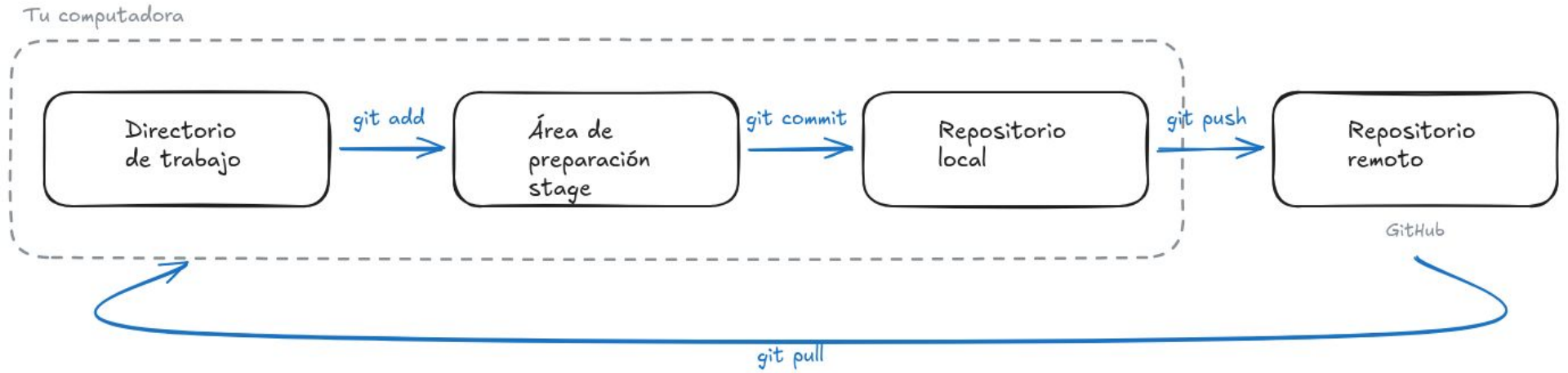
GitHub

- Servicio que aloja repositorios en la nube.
- Es el punto de encuentro del equipo.
- Alternativas: GitLab, Bitbucket.

Git es distribuido: cada integrante tiene una copia completa del proyecto y de todo su historial. El remoto no es "el original", es donde nos sincronizamos.

El camino de un cambio

Guardar en Git son dos pasos, no uno. Primero elijo qué cambios entran en la próxima fotografía (git add); después la saco y queda en mi historial (git commit); recién con git push sale de mi computadora.



El commit: una fotografía del proyecto

Qué guarda

- Un identificador único (hash).
- Autor y fecha: por eso configuramos nombre y correo.
- El estado del proyecto en ese momento.
- Un mensaje que explica el cambio.

Cómo se escribe el mensaje

- Agrega clase Estudiante con legajo y nombre
- Corrige el cálculo del promedio en Materia

En cambio: **"cambios", "asdasd", "ahora si"** no le dicen nada al equipo ni a ustedes mismos en tres semanas.

Un commit es una unidad de trabajo con sentido propio: ni medio archivo suelto, ni tres funcionalidades mezcladas.

Los comandos de hoy

- git clone <url> — traer el repositorio por primera vez
- git status — ¿en qué estado está todo?
- git add <archivo> — preparar un cambio
- git commit -m "mensaje" — confirmarlo en mi historial
- git push — publicarlo en el remoto
- git pull — traer lo que subieron los demás
- git diff — ver qué cambió, línea por línea
- git log --oneline --graph --all — ver el historial

Después de cada comando que modifique algo: **git status**. No cambia nada y siempre sugiere el próximo paso. Prohibidos por ahora: reset --hard, rebase y push --force.

Actividad: primer flujo de trabajo colaborativo

En parejas Estudiante A (dueño del repo) y Estudiante B (colaborador)

- Paso 0: verificar el entorno: git --version, git config --global --list, gh auth status
- Paso 1 : A crea el repositorio en GitHub e invita a B como colaborador
- Paso 2 : los dos clonan
- Paso 3 : A sube la estructura base; B la trae con git pull
- Paso 4 : los dos editan la misma línea, sin sincronizar
- Paso 5: resolver el conflicto, en pareja

Regla: git status después de cada comando, y leerlo.

Ahora ustedes

Actividad en parejas · la guía está en Classroom

¿Qué les dijo Git cuando B hizo push?

! [rejected] main -> main (fetch first)

- El commit de B estaba bien hecho, pero vivía únicamente en su repositorio local.
- Mientras tanto, A había publicado otro commit: el remoto avanzó por un camino distinto.
- Git no acepta un push que dejaría afuera commits que ya están en el remoto: no sobrescribe historia ajena.

Regla de equipo: pull antes de empezar a trabajar, push cuando terminás. La mayoría de los conflictos evitables nacen de no hacer pull al arrancar.

El conflicto: cuando los cambios se contradicen

Cuándo aparece

- Dos commits modifican la misma línea del mismo archivo.
- Git no puede decidir cuál versión es la correcta y frena la fusión.

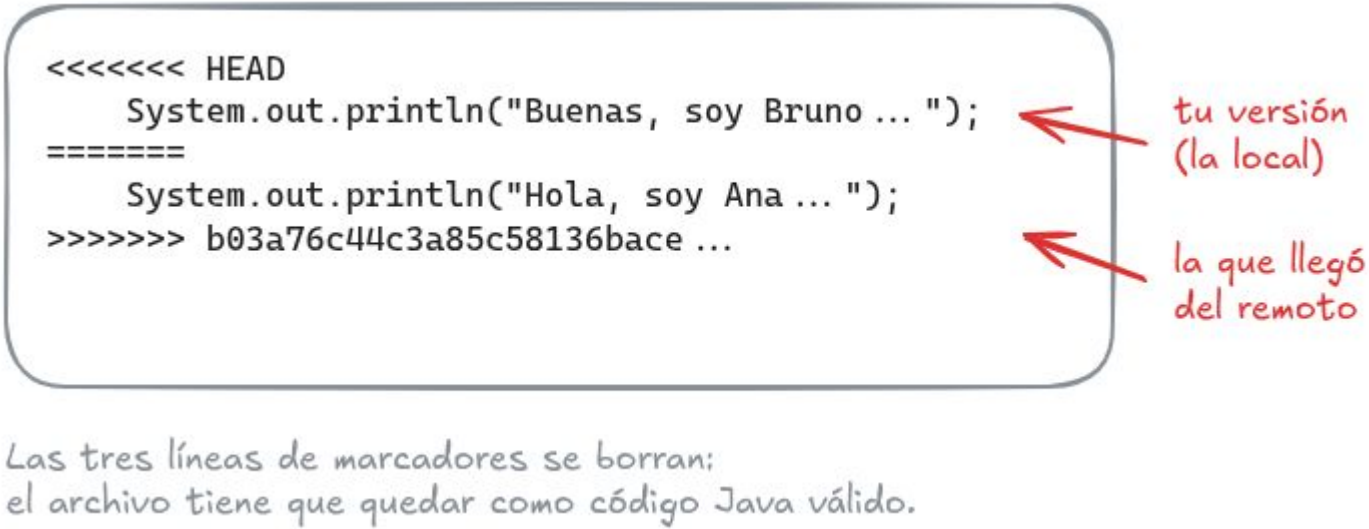
Cuándo NO aparece

- Archivos distintos, o líneas distintas del mismo archivo.
- Ahí Git fusiona solo y no hay que hacer nada.

Que Git frene no es una falla: es negarse a decidir por ustedes algo que solo ustedes pueden decidir.

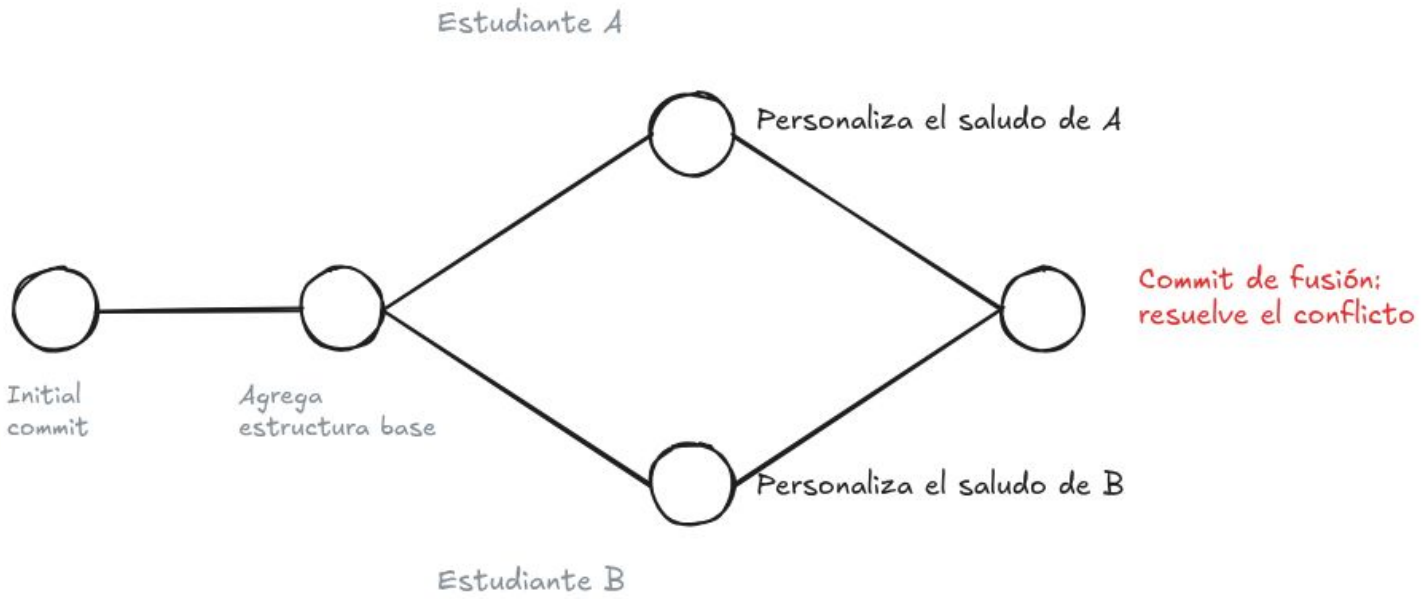
Resolver el conflicto es siempre manual

Editar, decidir qué queda, borrar los tres marcadores, y cerrar con **git add** + **git commit**. Ese último commit (el de fusión) es la prueba, en el historial, de que dos líneas de trabajo se unieron.



Lo que quedó en el historial

git log --oneline --graph --all muestra la forma real del trabajo: dos personas que se separaron y volvieron a unirse.



Ramas: hacer esto a propósito

- Hoy el desvío fue accidental. Una rama es exactamente eso, pero decidido.
- main contiene el código estable, el que compila y funciona.
- Para una funcionalidad nueva o un experimento se crea una rama y se commitea ahí, sin afectar a main.
- Cuando está terminado y probado, se fusiona (merge) de vuelta en main.

En el proyecto integrador: una rama por funcionalidad y main siempre compilando.

.gitignore: lo que no se sube

Qué queda afuera

- Compilados: *.class, bin/, out/, target/
- Configuración del IDE: .vscode/, .idea/
- Credenciales y contraseñas.

Por qué

- Se regeneran solos: no aportan historia.
- Cambian todo el tiempo y ensucian cada commit.
- Producen conflictos inútiles entre integrantes.
- Una contraseña subida a un repo público es pública para siempre.

La plantilla de Java de GitHub cubre los compilados, pero no las carpetas del IDE: hay que completarla.

Git en esta materia

- Un repositorio por grupo de proyecto integrador, desde la semana que viene.
- Las entregas parciales se hacen sobre ese repositorio: idea de desarrollo, diagrama de clases, herencia y polimorfismo, prototipo, patrones.
- El historial muestra el aporte de cada integrante: los commits son parte de la evidencia de trabajo.
- En la defensa oral individual se pregunta sobre el propio código y sobre las decisiones que quedaron registradas.

Hoy en la clase práctica: el mismo flujo con GitHub Desktop y VSCode, y armamos el repositorio del grupo.

Si no está commiteado y pusheado, no existe.